

CS5800: Algorithms — Virgil Pavlu

Homework 8

Name: SAI Vardhan Reddy Pathuri

Collaborators:

Instructions:

- Make sure to put your name on the first page. If you are using the $\text{\LaTeX}$ template we provided, then you can make sure it appears by filling in the `yourname` command.
- Please review the grading policy outlined in the course information page.
- You must also write down with whom you worked on the assignment. If this changes from problem to problem, then you should write down this information separately with each problem.
- Problem numbers (like Exercise 3.1-1) are corresponding to CLRS 3rd edition. While the 2nd edition has similar problems with similar numbers, the actual exercises and their solutions are different, so make sure you are using the 3rd edition.

1. (50 points)

Implement a hash for text. Given a string as input, construct a hash with words as keys, and word counts as values. Your implementation should include:

- a hash function that has good properties for text
- storage and collision management using linked lists
- operations: insert(key,value), delete(key), increase(key), find(key), list-all-keys

Output the list of words together with their counts on an output file. For this problem, you cannot use built-in-language data structures that can index by strings (like hashtables). Use a language that easily implements linked lists, like C/C++.

You can test your code on “Alice in Wonderland” by Lewis Carroll, at [link](#).

The test file used by TA will probably be shorter.

Try these three values for $m = \text{MAXHASH}$: 30, 300, 1000. For each of these m values, produce a histogram over the lengths of collision lists. You can also calculate variance of these lengths.

If your hash is close to uniform in collisions, you should get variance close to zero, and almost all list-lengths around $\alpha = n/m$.

If your hash has long lists, we want to know how many and how long, for example print the lengths of the longest 10% of the lists.

(Extra Credit) Find a way to record not only word counts, but also the positions in text. For each word, besides the count value, build a linked list with positions in the given text. Output this list together with the count.

Solution:

[Github repo](#).

2. (50 points)

Implement a red-black tree, including binary-search-tree operations *sort*, *search*, *min*, *max*, *successor*, *predecessor* and specific red-black procedures *rotation*, *insert*, *delete*.

You must implement delete with functionality for regular BST (without fixup procedure) but implementing delete for RB-trees (including fixup procedure) is Extra Credit.

Your code should take the input array of numbers from a file and build a red-black tree with this input by sequence of “inserts”. Then interactively ask the user for an operational command like “insert x” or “sort” or “search x” etc, on each of which your code rearranges the tree and if needed produces an output. After each operation also print out the height of the tree.

You can use any mechanism to implement the tree, for example with pointers and struct objects in C++, or with arrays of indices that represent links between parent and children. You cannot use any tree built-in structure in any language.

Solution:[Github repo](#)

3. (50 points)

Study the skiplist data structure and operations. They are used for sorting values, but in a datastructure more efficient than lists or arrays, and more guaranteed than binary search trees. Review Slides [skiplists.pdf](#) and [Visualizer](#). The demo will be a sequence of operations (asked by TA) such as for example insert 20, insert 40, insert 10, insert 20, insert 5, insert 80, delete 20, insert 100, insert 20, insert 30, delete 5, insert 50, lookup 80, etc

Solution: [Github repo](#)

4. (50 points)

Implement binomial heaps as described in class and in the book. You should use links (pointers) to implement the structure as shown in the figure 1. Your implementation should include the operations: Make-heap, Insert, Minimum, Extract-Min, Union, Decrease-Key, Delete

Make sure to preserve the characteristics of binomial heaps at all times: (1) each component should be a binomial tree with children-keys bigger than the parent-key; (2) the binomial trees should be in order of size from left to right. Test your code several arrays set of random generated integers (keys).

Solution:

[Github repo](#)

5. (25 points) Exercise 16.3-3

Solution:

Let n denote the current number of elements in the heap. the potential function

$$\Phi(H) = c \cdot n \lg n$$

for $c > 0$.

Amortized cost definition

For an operation i with actual cost c_i and heap size changing from n_i to n_{i+1} , the amortized cost is

$$\hat{c}_i = c_i + \Phi(H_{i+1}) - \Phi(H_i)$$

Amortized cost of INSERT

Actual cost:

$$c_{\text{insert}} = O(\lg n)$$

After insertion, the heap size increases from n to $n + 1$. Hence,

$$\Phi(H_{i+1}) - \Phi(H_i) = c[(n+1)\lg(n+1) - n\lg n]$$

$$(n+1)\lg(n+1) - n\lg n = \lg n + O(1)$$

$$\Phi(H_{i+1}) - \Phi(H_i) = c \lg n + O(1)$$

So the amortized cost of INSERT is

$$\hat{c}_{\text{insert}} = O(\lg n) + O(\lg n) = O(\lg n)$$

Amortized cost of EXTRACT-MIN

Actual cost:

$$c_{\text{extract}} = O(\lg n)$$

After extraction, heap size decreases from n to $n - 1$. Hence,

$$\Phi(H_{i+1}) - \Phi(H_i) = c[(n-1)\lg(n-1) - n\lg n]$$

$$(n-1)\lg(n-1) - n\lg n = -\lg n + O(1)$$

so

$$\Phi(H_{i+1}) - \Phi(H_i) = -c \lg n + O(1)$$

Therefore, the amortized cost of EXTRACT-MIN is

$$\hat{c}_{\text{extract}} = O(\lg n) - c \lg n + O(1)$$

By choosing c sufficiently large the $\lg n$ terms cancel, yielding

$$\hat{c}_{\text{extract}} = O(1)$$

Conclusion

With the potential function

$$\Phi(H) = c \cdot n \lg n, \quad c > 0$$

we obtain the desired amortized time bounds:

INSERT: $O(\lg n)$ amortized time EXTRACT-MIN: $O(1)$ amortized time

Intuition

The intuition is that each INSERT operation increases the heap size and thereby increases the potential. In contrast, each EXTRACT-MIN operation decreases the heap size and releases the stored potential, effectively making it logarithmic.

Solution:

Solution:

Solution:

Solution: